



**Design and Implementation of an eBPF-based Runtime Security
Monitoring Tool**

Candidate: Simone Romano (s344024)
Supervisor: Prof. Riccardo Sisto
Company tutor: Dr. David Soldani
Academic year: 2025–2026

Summary

Context and objective. Modern Linux hosts execute application workloads alongside container runtimes, orchestration services and administrative processes. Although containers separate workloads at user-space level, their processes still depend on the same host kernel for execution, credentials, files, memory and isolation. This makes the kernel a useful observation point for activity that crosses application and container boundaries.

Application logs expose only the information selected by their developers and may omit operations performed through libraries, helper programs or compromised processes. Kernel observations can provide a more uniform view, but their meaning depends on where an operation is observed. A system-call entry records what a process requested, a security hook exposes an operation under mediation, and a later transition or return value may establish what occurred. A useful monitor must preserve these distinctions while retaining enough process and resource context to relate the resulting evidence.

Collecting this information also introduces practical constraints. A host produces continuous background activity, so transporting every available event can increase CPU and memory consumption while obscuring the operations relevant to an investigation. Moreover, an enterprise kernel may provide vendor backports and data layouts that differ from the corresponding upstream release. The monitoring design must therefore combine selective collection with bounded processing and compatibility verified on the actual target system.

This thesis designs, implements and evaluates a node-local runtime security monitor based on eBPF and a Go user-space runtime. The prototype selectively collects host events, converts them into a common typed representation and evaluates them through configurable policies and deterministic detectors. The reference environment is Rocky Linux 8.10 with an enterprise Linux 4.18 kernel. BTF and CO-RE are used to adapt access to kernel data structures, while explicit compatibility paths address target-specific differences. The prototype observes and detects activity without modifying kernel decisions.

Architecture and contributions. The central contribution is an end-to-end monitoring pipeline that combines host-security observation points with a stable binary event contract, selective loading, early filtering and local detection. Logical events form the public interface, while a probe registry maps them to the physical eBPF programs and internal dependencies required for collection. This separation allows several hooks or system-call variants to produce one event schema without exposing implementation details to policies and detectors.

Kernel operations are observed at complementary stages. System-call entry preserves the caller's request, Linux Security Module hooks expose security mediation, scheduler and credential hooks report transitions that reached kernel state, and return probes add the result visible at the observed boundary. Direct producers construct an event in one invocation; paired producers retain entry data per thread and event, then complete it at return. Per-CPU workspaces keep large records off the eBPF stack, and indexed arguments allow unavailable fields to be omitted without changing the meaning of later values. Only the populated record prefix is submitted through the perf buffer.

The implemented hooks cover process lifecycle and executable transitions, credentials and process control, file and memory activity, namespaces, cgroups, modules and selected kernel-facing interfaces. Shared helpers provide event initialisation, bounded serialisation and an optional exact effective-UID filter that can reject out-of-scope observations before enrichment and transport.

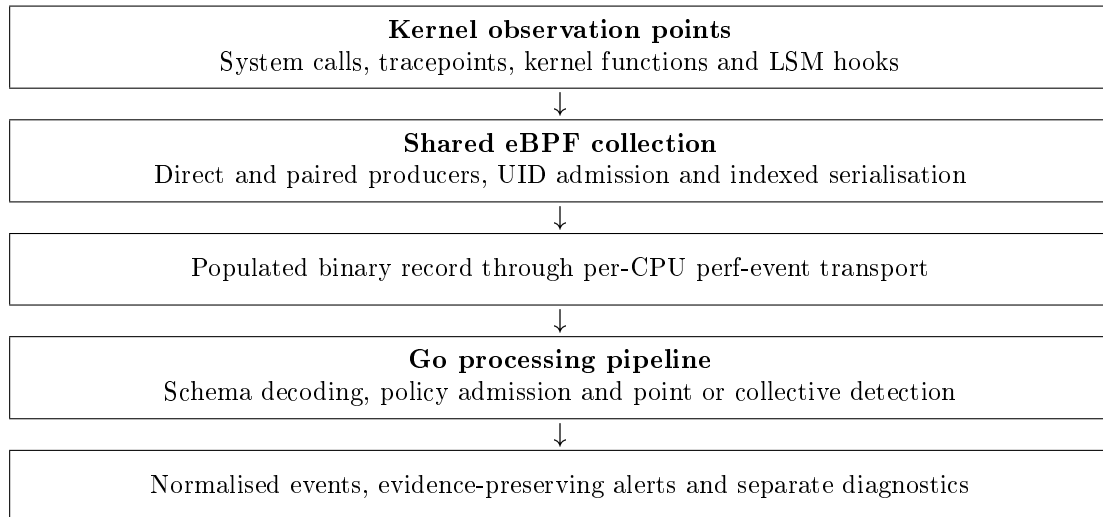


Figure 1: End-to-end path from kernel observation to user-space results.

User-space processing and detection. The Go runtime validates the configuration, resolves the selected logical events and expands their probe dependencies before loading the eBPF object. The decoder interprets the common context and indexed arguments through the event registry, producing the typed representation shared by downstream components. Policies define the admitted event domain and suppression rules; detectors evaluate the resulting evidence and generate alerts.

Point detectors match conditions over one event. Collective detectors retain bounded partial state for short ordered sequences and correlate observations through stable process identity, immediate process relationships, local device–inode identity, cgroup identity or configured combinations of these dimensions. Alerts preserve the matching observations in order and may carry MITRE ATT&CK tactics and techniques as classification metadata. Normalised events and alerts are written separately from operational diagnostics, so runtime failures are not confused with monitored activity.

Experimental evaluation. The prototype was evaluated on a two-core Minikube node using a vulnerable CTF workload. The monitor, target and triggering client ran in separate Pods on the same node, allowing the experiment to verify cross-Pod visibility through their shared host kernel. Three benign runs completed without an alert. In all three exploit runs, the client recovered the flag and the detector produced exactly one collective alert containing four ordered observations: execution of a shell, execution of *cat*, resolution of the flag file and the successful *open* result.

Performance was measured at the monitor-container cgroup boundary. With exact UID admission enabled, mean CPU consumption was 0.46% of one core, compared with 5.73% for the same monitoring configuration without the filter. The observed reduction was approximately 92%, and the filtered profile remained below the project’s 5% acceptance threshold. Peak cgroup memory was 92.17 MiB with filtering and 110.21 MiB without it. Falco and Tetragon were evaluated in separate three-run campaigns on the same Minikube testbed and against the same exploit, both with and without a UID condition. In the filtered profiles, the prototype achieved the lowest mean CPU consumption: 0.46% of one core, compared with 1.88% for Tetragon and 5.15% for Falco, demonstrating the effectiveness of its kernel-side UID filter.

Conclusion and boundaries. This work shows that useful runtime visibility depends not only on collecting kernel events, but on preserving where each observation occurs and how it relates to the surrounding activity. By distinguishing requests, mediation points and outcomes, then correlating their evidence in user space, the prototype turns heterogeneous kernel observations into interpretable security alerts. This provides a concrete basis for extending node-level monitoring without coupling detection logic to individual kernel hooks.